Execution-Preserving Specification Tests with QCP

Instantiation, Normalization, Inlined Calls, and Observational VCs

TeSpec implementation and semantics note

October 10, 2026

Abstract

TeSpec tests partially instantiated separation-logic specifications by executing the corresponding C bodies with QCP's symbolic executor. Assertions, loop invariants, and function contracts generate verification conditions on copies of the current logical state. Actual loop iterations and function bodies determine the successor state. We specify argument transfer, fresh activation records, shared heap updates, return matching, and continuous normalization. The soundness statement is relative to QCP's supported C semantics, its memory and VC rules, and the trusted decision procedures. An implementation-to-rule mapping separates this argument from regression evidence and from independently checked proof certificates.

1 Inputs and semantic scope

A concrete state is $\sigma = (\rho, h)$ and a logical valuation is η. Assertions use QCP's memory model; separating conjunction partitions the heap, whereas ordinary conjunction shares it. A contract has the form

$$\{R(a_0, w)\}\ f(a)\ \{E(a_0, w, r)\}.$$

Here a_0 denotes immutable entry argument values, w denotes logical `With` parameters, and r denotes a return value. A typed partial input substitution θ_0 binds logical identities. Pointer addresses may remain symbolic while logical contents and sizes are bound to constructor terms. Mutating a C parameter or a pointed-to cell does not modify its immutable entry value.

There are two distinct obligations:

$$\begin{aligned} \mathsf{SAT}_0 &\equiv \exists \eta, \rho, h.\ R[\theta_0](\rho, h, \eta), \\ \mathcal{T} &\equiv \{R[\theta_0]\}\ \mathit{body}\ \{E[\theta_0]\}. \end{aligned}$$

A satisfiability witness is not silently adopted as the execution input. Unbound symbols range over all allowed interpretations during execution. Thus a fixed list length need not determine its node addresses or contents.

2 One execution state

A configuration is

$$C = \langle pc, P, K, \theta_0, \Omega, b, m \rangle.$$

P is a QCP assertion containing pure facts, a program-variable/address map, spatial resources, and binders; K is the activation stack, Ω contains obligations, and b contains execution budgets. There is no second mutable instance store. An optional trace observes P and does not participate in execution.

Both compatibility modes execute loop iterations and available function bodies. `expand` omits intermediate annotation/contract checks by default; `no-expand` enables them. Explicit `check-preserve` also enables them in the first mode. The entry Require, operational safety, and final Ensure are checked in both. The mode names therefore identify checking policies, not different operational semantics.

For an enabled boundary, let A_P^K be the annotation resolved against the current path and invocation. The transition is

$$\langle pc, P, K, \theta_0, \Omega, b, m \rangle \longrightarrow \langle pc^+, P, K, \theta_0, \Omega \cup \{\mathrm{copy}(P) \vdash A_P^K\}, b, m \rangle.$$

The exact same execution assertion is returned. Failure to close a boundary VC does not authorize adding its assertion to the path condition. Actual C branches may add their evaluated guards.

3 Instantiation and continuous normalization

Substitution is typed, keyed by logical identity, and capture avoiding. Invocation variables and local binders are freshened before substitution. Replacing a bound logical parameter removes the corresponding binder before introducing a term that may mention an existing symbol.

The normalization contract is

$$\Gamma \models F \iff \exists z.\mathsf{N}_\Gamma(F),$$

where z contains fresh variables introduced by definition expansion. Γ consists solely of facts available on the current path. Known arithmetic, Boolean and list operations are evaluated, and available logical definitions are unfolded. Unknown terms remain residual; recursive unfolding retains all feasible alternatives and stops at its resource budget.

Normalization must also preserve correlations with externally held values. Let

$$\mathsf{Obs}(F,t) = \{(\sigma, v) \mid \exists \eta.\ \sigma, \eta \models \mathit{body}(F) \wedge v = [\![t]\!]_\eta\},$$

where one valuation jointly interprets the existential witnesses and the result term. If t is retained, normalization requires $\mathsf{Obs}(\mathsf{N}(F), t) = \mathsf{Obs}(F, t)$. Eliminating r from $\exists r.\ r = 2 \wedge \mathit{cell}(p, 1)$ while retaining r as the result term violates this condition, despite preserving the closed assertion's satisfiability. The implementation protects all existential identities present before a predicate-expansion call. Only fresh local witnesses introduced by that call may be eliminated; branch-local constant propagation retains its premise equalities. This preserves return, expression-stack, and frame correlations without introducing a mutable instance cache.

A statement transition reuses QCP:

$$\big((P_1, \ldots, P_k), \Omega_e\big) = \mathsf{QCP.Exec}(c, \mathsf{N}(P)), \qquad \mathsf{succ}(C) = \{\mathsf{N}(P_i)\}_{i=1}^{k}.$$

The executor accumulates Ω_e. A branch can be discarded only when its infeasibility is established. Typed C evaluation and its safety obligations remain responsible for overflow, division, shifts, initialization and memory access. Mathematical simplification cannot remove those requirements. Predicate folding and unfolding must preserve the represented heap.

4 The inlined-call algorithm

4.1 Evaluate first, then allocate a frame

For $f(e_1, \ldots, e_n)$, QCP evaluates the target and arguments into a family

$$\mathsf{EvalArgs}(P, f, e_1, \ldots, e_n) = \{(P_j, f_j, \vec{v}_j, \Omega_j)\}_j.$$

Each member is processed independently. Values are passed after evaluation; argument expressions are not evaluated again after callee writes. The function body is resolved by frontend function identity, including the connection between declarations and definitions. An unresolved target or missing body cannot be simulated using a contract successor.

4.2 Reuse QCP matching to prepare checking goals

When boundary checking is enabled, run QCP's normal contract application on copy(P_j) with the selected contract and explicit logical/type arguments. The `BOUNDARY_CALL` adapter reuses its freshening, argument substitution, separation matching, and VC constructors. It obtains an entry substitution ω and frame F, with obligation

$$P_j \vdash R[\vec{v}_j/a_0, \omega/w] * F.$$

The candidate result

$$G = E[\vec{v}_j/a_0, \omega/w, r] * F$$

is retained only as a return-checking goal. The same ω is used in Require and Ensure. Existentials declared by Ensure itself remain existential at each return path. Failure to determine the binding is recorded as `contract_binding_unknown`; it cannot count as a passed boundary. Some unsupported With/type filling paths currently stop execution with UNKNOWN. The common matcher-failure path can continue executing the body.

The checking adapter preserves the relationship between candidate return terms and their assertions. It does not eliminate a return binder before the actual return equality is attached. No contract successor is installed in P_j.

4.3 Fresh storage and shared pointer values

Let κ be a fresh activation. For scalar and pointer parameters, allocate fresh slots α_i and set

$$P_{\text{entry}} = P_j * \overset{n}{\underset{i=1}{*}}(\alpha_i \mapsto_{\tau_i} v_{j,i}), \qquad \mathsf{Local}_\kappa(a_i) = \alpha_i.$$

Freshness applies to the parameter slots. The values $v_{j,i}$, including pointer identity and aliases, are preserved. Pointer reassignment affects the private slot, while a dereference modifies the shared pointed-to resource.

A struct value parameter uses QCP's typed record-copy rule into fresh storage, preserving its field values and undefined-field information. Pointer fields retain their shared targets. A currently unsupported multi-result parameter copy is UNKNOWN; by-value union parameters are also outside the implemented adapter.

The frontend may reuse a source-level program-variable ID during recursion. The runtime map must not reuse its address. The activation saves shadowed mappings, imports formals, and creates fresh local storage at declarations. It clones the PSTree runtime control state, resets iteration counts, and remaps return, break, continue and goto targets into the invocation tree.

4.4 Boundaries inside a body

Each visited annotation is cloned before parsing. Its program variables are resolved through the current invocation's address map. Its logical With parameters use the immutable entry substitution ω. A function-local assertion is checked with the caller frame obtained by QCP matching. These operations modify the checking goal only.

Disjunctive source states are checked separately. This prevents an annotation from retaining the constant or address obtained when another branch or an earlier invocation first visited the same source text. Which-implies annotations retain their logical implication obligations without changing execution state.

4.5 Return all feasible results

QCP evaluates each actual return expression, producing (P_k, r_k). The adapter retains all return alternatives, including early returns. A completed non-void body with no returned value is UNKNOWN rather than an arbitrary synthesized value.

The operational scope rules remove local storage belonging to this activation. Parameter cleanup uses the fresh parameter addresses and QCP's type-specific deallocation rules. Shared writes survive. Struct returns use QCP's temporary record storage so that caller-side copying outlives the callee's local record. Caller variable mappings, environment and control cursor are then restored, yielding P_k^-.

For candidate post branches G_q with existential variables u_q and return terms r_q, generate, for every actual return,

$$P_k^- \vdash \bigvee_q \exists u_q.\big(G_q \wedge r_q = r_k\big).$$

For a contract without a return term, omit the equality. Keeping it as an equality supports constant, variable and existential return descriptions. The adapter uses QCP's `CheckEntailment` and emits its usual auto/manual VCs. It resumes the caller with (P_k^-, r_k) regardless of the candidate goal.

4.6 Pseudocode

```
Inline(P, function, arguments, policy, budget):
  results := []
  for (Q, f, values, evalVCs) in QCP.EvalCall(P, arguments):
    require a resolved body and available budget
    goals := none
    if policy checks contracts and f has a selected contract:
      goals, entryVCs := QCP.ContractCheck(copy(Q), values)
      record entryVCs or incomplete binding
    save caller environment and local mappings
    allocate fresh parameter storage; copy struct values
    execute a fresh runtime tree using QCP statement rules
    for each actual (T, value) returned by the body:
      U := clean this frame's local/parameter storage in T
      restore caller mappings in U
      if goals available: emit QCP.Entail(U, goals(value))
      append (U, value) to results
```

```
    restore caller cursor and environment
  return all results and accumulated VCs
```

4.7 Example: a weak postcondition preserves a concrete write

Suppose `bump(p)` requires $p \mapsto v$ and ensures $\exists w.\, p \mapsto w$, with body `*p = *p + 1`. From $P_0 \mapsto 3 * Q_0 \mapsto 8$, calling `bump(`$P_0$`)` selects $v = 3$ and frame $Q_0 \mapsto 8$. A fresh parameter slot contains P_0, and the real write produces $P_0 \mapsto 4$. After cleanup, the successor is $P_0 \mapsto 4 * Q_0 \mapsto 8$. The Ensure VC chooses $w = 4$; the successor retains the value 4.

5 Loops and incomplete executions

At each visited invariant location, emit $P_t \vdash \mathit{Inv}$ if enabled, then evaluate the actual guard using P_t. Execute true branches and collect false branches as exits. Continue, break, for-updates and do-while tests follow QCP's control-flow representation. No invariant replaces P_t or supplies an arbitrary loop-head state. These are checks of states reached by the input family, rather than an inductive proof for all possible inputs.

Loop, call-depth and wall-clock budgets apply to both modes. Exhaustion yields UNKNOWN and preserves available diagnostics. A finite prefix is not a completed execution or a nontermination certificate. A declaration-only test target may receive a specification-only Require check; its execution and Ensure phases are NOT_RUN. A reached external call with a contract but no body is UNKNOWN. A call with neither body nor contract is an error.

6 Require satisfiability and verdicts

The normalized instantiated Require is handled by the existing precondition checker. SAT requires a complete encoding or an established rule for the fragment. UNSAT may follow from contradictory bindings, overlapping separated cells or arrays, or known range constraints. Abstract recursive predicates and unresolved layout requirements retain existential ITP VCs. Spatial predicates are not universally assumed satisfiable.

Execution-scope PASS requires initial SAT, completed coverage of every feasible path, all selected VCs discharged, and no unreported unsupported operation or budget exhaustion. INVALID denotes an unsatisfiable input Require. UNKNOWN covers unresolved obligations and incomplete execution. Parser/executor errors remain separately visible and never count as PASS. The automatic stage launches no proof agent. Reports distinguish the trusted solver decision from an independently checked certificate; without the latter, the certificate field remains NOT_CHECKED.

7 Relative soundness

Lemma 1 (Boundary noninterference)**.** *For fixed P and a resolvable boundary, enabling the check leaves the operational successor assertion unchanged. It only adds obligations.*

Proof. Parsing, matching and VC generation operate on independent copies and checking goals. The boundary returns the original assertion. Its result is never used to filter a path or install a successor. □

Lemma 2 (Invocation isolation)**.** *Fresh parameter/local addresses, shared argument pointer values, and activation-specific cleanup preserve caller storage identity while retaining all callee writes to shared memory.*

Proof. Fresh slots are separated from existing storage. Each source variable resolves through its current activation mapping. Callee assignments therefore target its own slots or the shared locations denoted by their stored values. Cleanup removes only storage owned by that activation; saved caller mappings are restored. QCP's typed copy rules provide the corresponding value-transfer property for supported structs. □

Theorem 1 (Relative execution-scope soundness)**.** *Assume sound QCP primitive transitions, memory and VC rules for the supported C subset; semantics-preserving normalization; valid discharged obligations; and complete preservation of feasible symbolic branches. If TeSpec returns execution-scope PASS, every execution from a state satisfying $R[\theta_0]$ is covered by the completed symbolic execution, is safe under those semantics, and satisfies $E[\theta_0]$ and all enabled visited boundaries.*

Proof argument. Induct over operational steps and activation depth. Initialization follows from typed instantiation. QCP primitive soundness preserves coverage, and normalization preserves assertion interpretation. Branching retains all feasible successors. Boundary noninterference preserves the induction hypothesis. Invocation isolation establishes the callee entry relation; the induction hypothesis covers its body, and cleanup/return restoration gives the caller successor. Return entailments establish the selected postconditions. Loops are repeated operational steps. An exhausted budget cannot satisfy the premise that PASS was returned. Initial satisfiability rules out vacuous success from an inconsistent input. □

This is an algorithm-level argument with explicit dependencies. Regression and sanitizer tests support the implementation; a mechanized Coq metatheory of the complete implementation has not been produced.

8 Implementation and reproducible evidence

Component	Reused or adapted implementation
Argument evaluation and result stack	QCP `ExprExec.c`
Fresh activation and control cloning	`ConcreteCall.c`
Require matching, With and frame	`FuncCallExec`, `FuncCallWit`
Actual return versus Ensure	`CheckInlineReturn`, `CheckEntailment`
Record copies and annotation goals	`SymExecutor.c`
Actual loop/control steps	`StateMachine.c`
Continuous normalization	`ExpressionNormalize.c`, `ExpressionPredicate.c`
Initial existence	`PreconditionCheck.c`
Binary integration and resources	teSpec deterministic controller

The current evidence manifest is `instance-semantics-evaluation.json`. Positive/negative semantic regressions, curated QCP inputs, and full-corpus synthetic-input scans have distinct roles. Synthetic sizes 0, 1 and 2 are not a correctness oracle; INVALID inputs are counted separately. Unsupported features and errors remain visible. No implementation dispatch is based on an example filename or function name.

9 Categorized end-to-end evaluation

The executable matrix contains 52 functional programs and 37 diagnostic scenarios in 18 categories: scalar control, integer operations, loops, recursion, calls, arrays, strings, pointer aliases, structures, linked structures, trees/DAGs, allocator patterns, defined predicates, VC boundaries, preconditions, C safety, incomplete executions, and declarations. Each functional program has four inputs, three variants (reference, wrong postcondition, and implementation mutation), and two modes: 1,248 runs. The diagnostic scenarios contribute 74 runs. Independent Python results are cross-checked against native C returns and heap observations; every mutation must be distinguished by at least one native input.

All 1,322 runs meet their expectations, including 779 negative checks and zero false acceptances. The verdict distribution is 541 PASS, 757 UNKNOWN, 14 ERROR, and 10 INVALID. Deliberately wrong postconditions retain return VCs and therefore contribute UNKNOWN rather than PASS. The same matrix meets its expectations under ASan/UBSan with leak detection disabled. Earlier runs exposed eight false acceptances caused by the return-binding elimination described above; the protected-binding rule removes them. The accompanying JSON records binary/source identities and artifact paths. ITP certificates were not replayed in this evaluation.

Fault classes follow the partial-specification and allocator experiments in Sections 4.3–4.5 of *Code-Specify-Test-Debug-Prove*, whose official artifact is available at Zenodo. Our programs are independently authored. Finite symbolic-address shapes and selected native instances define this matrix's scope; the separate QCP corpus scan records unresolved abstract predicates and unsupported program features.